PRACTICAL 3

DATA INGESTION PIPELINE COMPARISON

1. AIM

To implement and compare **batch ingestion** and **real-time streaming ingestion** using Python and Redpanda, and evaluate their latency, throughput, scalability and fault tolerance.

2. PROBLEM STATEMENT

Implement two ingestion approaches for simulated user activity data:

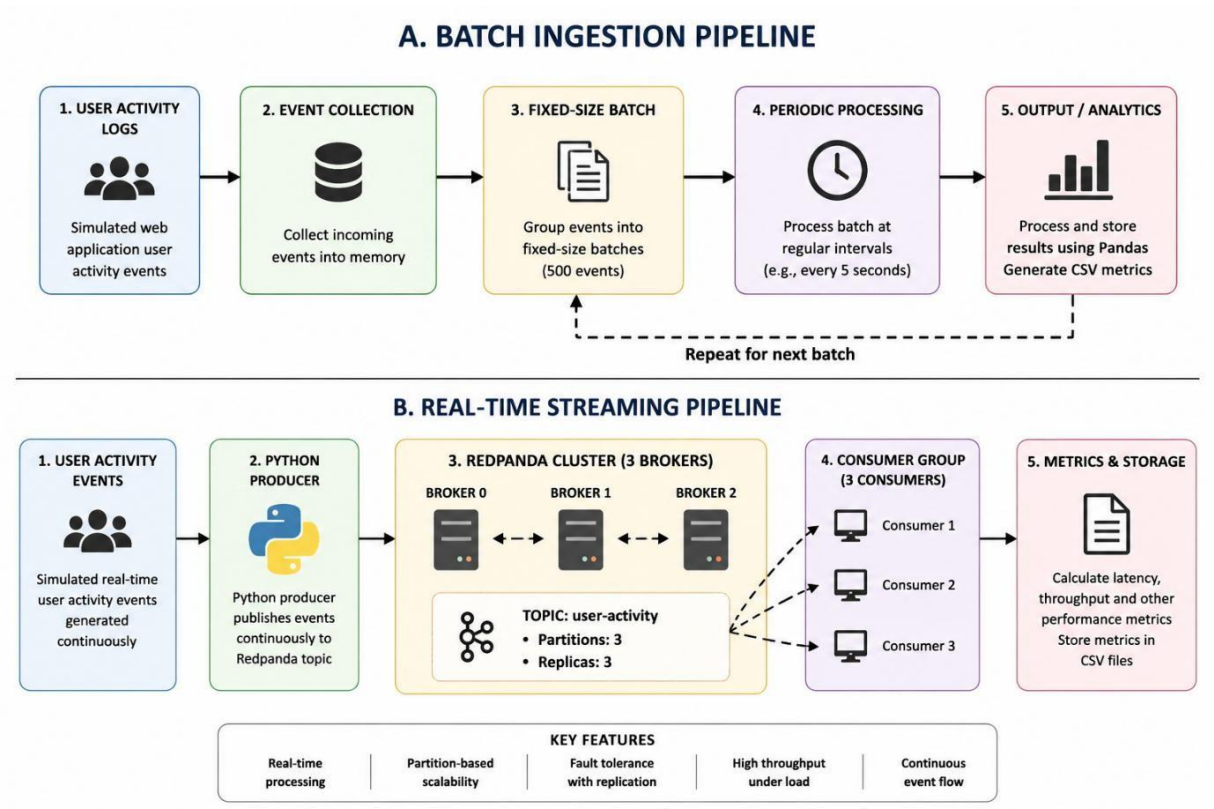
- **Batch ingestion:** Events are collected and processed in fixed-size batches.
- **Real-time streaming:** Events are continuously sent to and consumed from a Redpanda topic.

The pipelines are tested under normal load, traffic spikes and broker failure.

3. TECHNOLOGIES USED

Technology	Purpose
Python	Producer, consumer and batch processing
Redpanda	Event streaming/message broker
Docker	Running Redpanda brokers
Pandas	Data processing and metrics
PowerShell	Running the practical
CSV	Storing metrics

4. SYSTEM ARCHITECTURE



5. BATCH INGESTION

The batch pipeline processes events in groups of **500**.

For the experiment:

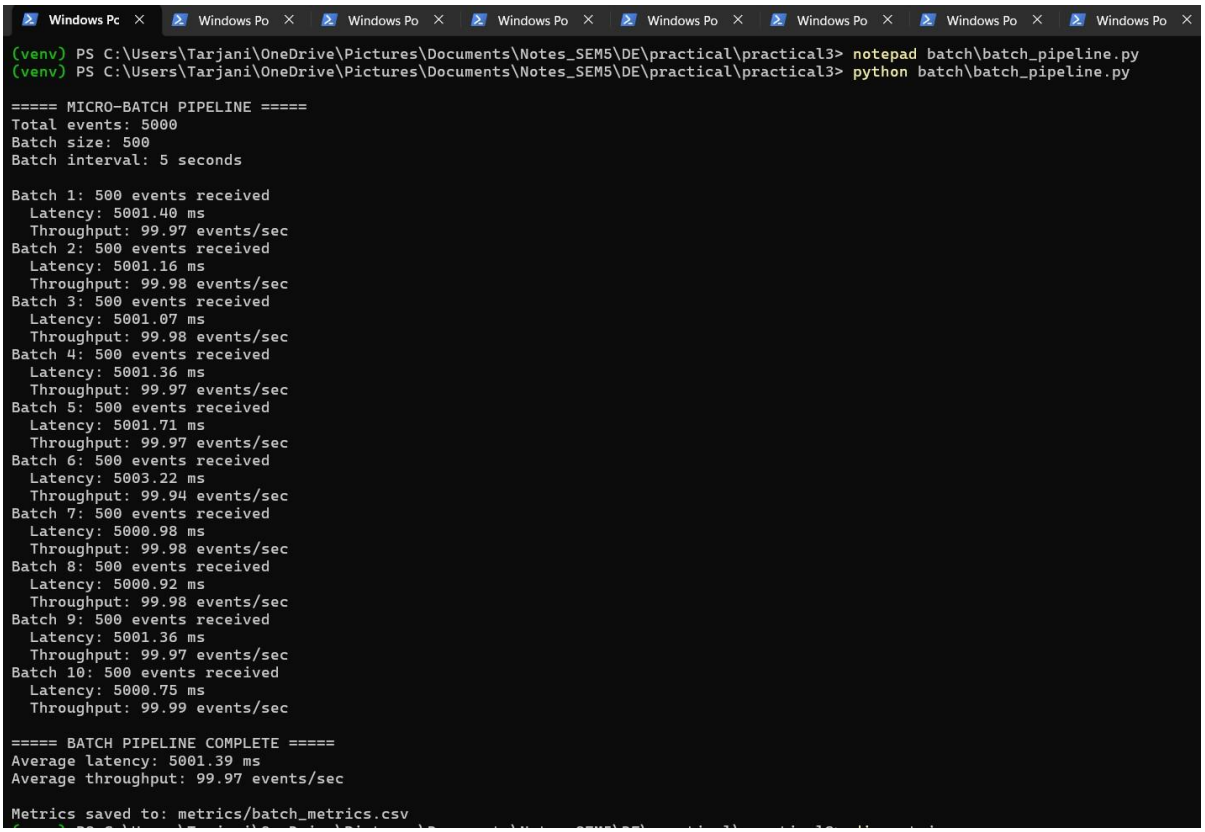
- Total events = **5,000**
- Number of batches = **10**
- Batch size = **500**

The metrics were stored in:

metrics/batch_metrics.csv

Batch Result

Metric	Value
Total events	5,000
Batches	10
Average response time	418.795 ms
Average processing time	0.589 ms
Average latency	5,001.391 ms
Average throughput	99.973 events/sec
Errors	1,857



6. REAL-TIME STREAMING

The streaming pipeline continuously sends events from a Python producer to the Redpanda user-activity topic.

The topic was configured with:

- **3 partitions**
- **3 replicas**

The normal streaming test processed **5,000 events**.

```
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practic
al\practical3> docker exec redpanda-0 rpk topic describe user-activity
SUMMARY
=======
NAME        user-activity
PARTITIONS  3
REPLICAS    3

CONFIGS
=======
KEY                                VALUE          SOURCE
cleanup.policy                     delete         DEFAULT_CONFIG
compression.type                   producer       DEFAULT_CONFIG
delete.retention.ms                -1             DEFAULT_CONFIG
flush.bytes                        262144         DEFAULT_CONFIG
flush.ms                           100            DEFAULT_CONFIG
initial.retention.local.target.bytes -1           DEFAULT_CONFIG
initial.retention.local.target.ms  -1             DEFAULT_CONFIG
max.compaction.lag.ms              9223372036854  DEFAULT_CONFIG
max.message.bytes                  1048576        DEFAULT_CONFIG
message.timestamp.type             CreateTime     DEFAULT_CONFIG
min.cleanable.dirty.ratio          0.2            DEFAULT_CONFIG
min.compaction.lag.ms              0              DEFAULT_CONFIG
redpanda.iceberg.mode              disabled       DEFAULT_CONFIG
redpanda.leaders.preference        none           DEFAULT_CONFIG
redpanda.remote.allowgaps          false          DEFAULT_CONFIG
redpanda.remote.delete             true           DEFAULT_CONFIG
redpanda.remote.read               false          DEFAULT_CONFIG
redpanda.remote.write              false          DEFAULT_CONFIG
retention.bytes                    -1             DEFAULT_CONFIG
retention.local.target.bytes       -1             DEFAULT_CONFIG
retention.local.target.ms          86400000       DEFAULT_CONFIG
retention.ms                       604800000      DEFAULT_CONFIG
segment.bytes                      134217728      DEFAULT_CONFIG
segment.ms                         1209600000     DEFAULT_CONFIG
write.caching                      true           DEFAULT_CONFIG
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practic
al\practical3>
```

```
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_
ractical\practical3> python streaming\consumer.py

===== STREAMING CONSUMER =====
Topic: user-activity
Consumer group: streaming-consumer-group
Target events: 5000

Processed 500 events | Throughput: 3.34 events/sec
Processed 1000 events | Throughput: 6.67 events/sec
Processed 1500 events | Throughput: 10.01 events/sec
Processed 2000 events | Throughput: 13.34 events/sec
Processed 2500 events | Throughput: 16.67 events/sec
Processed 3000 events | Throughput: 20.00 events/sec
Processed 3500 events | Throughput: 23.33 events/sec
Processed 4000 events | Throughput: 26.65 events/sec
Processed 4500 events | Throughput: 29.97 events/sec
Processed 5000 events | Throughput: 33.29 events/sec

===== STREAMING CONSUMER COMPLETE =====
Events processed: 5000
Average latency: 17.719 ms
Throughput: 33.28 events/sec
Metrics saved to: metrics/streaming_metrics.csv
```

7. TRAFFIC SPIKE AND SCALING

A traffic spike of **15,000 events** was generated and processed using three consumers in the same consumer group.

Consumer	Events
Consumer 1	4,972
Consumer 2	4,994
Consumer 3	5,034
Total	**15,000**

Average latency:

22.635 ms

```
PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practical\prac
tical3> .\venv\Scripts\Activate.ps1
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practic
al\practical3> $env:CONSUMER_ID="consumer-1"
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practic
al\practical3> python streaming\spike_consumer.py

===== SPIKE CONSUMER =====
Consumer ID: consumer-1
Consumer group: spike-consumer-group
Idle timeout: 5 seconds
Waiting for traffic...


===== SPIKE CONSUMER COMPLETE =====
Consumer ID: consumer-1
Events processed: 4972
Average latency: 23.414 ms
Throughput: 46.03 events/sec
Metrics saved to: metrics/consumer-1_spike_metrics.csv
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practic
```

```
PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practical\prac
tical3> .\venv\Scripts\Activate.ps1
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practic
al\practical3> $env:CONSUMER_ID="consumer-2"
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practic
al\practical3> python streaming\spike_consumer.py

===== SPIKE CONSUMER =====
Consumer ID: consumer-2
Consumer group: spike-consumer-group
Idle timeout: 5 seconds
Waiting for traffic...


===== SPIKE CONSUMER COMPLETE =====
Consumer ID: consumer-2
Events processed: 4994
Average latency: 20.820 ms
Throughput: 68.23 events/sec
Metrics saved to: metrics/consumer-2_spike_metrics.csv
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practic
al\practical3>
```

```
PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practical\prac
tical3> .\venv\Scripts\Activate.ps1
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practic
al\practical3> $env:CONSUMER_ID="consumer-3"
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practic
al\practical3> python streaming\spike_consumer.py

===== SPIKE CONSUMER =====
Consumer ID: consumer-3
Consumer group: spike-consumer-group
Idle timeout: 5 seconds
Waiting for traffic...


===== SPIKE CONSUMER COMPLETE =====
Consumer ID: consumer-3
Events processed: 5034
Average latency: 23.666 ms
Throughput: 118.72 events/sec
Metrics saved to: metrics/consumer-3_spike_metrics.csv
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practic
al\practical3>
```

8. FAULT TOLERANCE

One Redpanda broker, redpanda-1, was stopped while processing the 15,000-event workload.

```
PS C:\Users\Tarjani> docker stop redpanda-1
redpanda-1
PS C:\Users\Tarjani> docker ps
CONTAINER ID   IMAGE                                                  COM
MAND                  CREATED        STATUS        PORTS

          NAMES
8387e1fbfd2d   docker.redpanda.com/redpandadata/redpanda:v25.2.1   "/e
ntrypoint.sh redp…"   21 hours ago   Up 21 hours   0.0.0.0:18082->1808
2/tcp, [::]:18082->18082/tcp, 0.0.0.0:19092->19092/tcp, [::]:19092->19
092/tcp   redpanda-0
c7aa62076728   docker.redpanda.com/redpandadata/redpanda:v25.2.1   "/e
ntrypoint.sh redp…"   21 hours ago   Up 21 hours   0.0.0.0:18084->1808
4/tcp, [::]:18084->18084/tcp, 0.0.0.0:19094->19094/tcp, [::]:19094->19
094/tcp   redpanda-2
PS C:\Users\Tarjani> docker exec redpanda-0 rpk cluster info
CLUSTER
=======
redpanda.da47e92f-4760-47db-bb3f-b3c6f7f9f1da

BROKERS
=======
ID    HOST        PORT
0*    redpanda-0  9092
2     redpanda-2  9092

TOPICS
======
NAME                PARTITIONS  REPLICAS
__consumer_offsets  3           3
user-activity       3           3
```

The result was:

Metric	Value
Events sent	15,000
Events processed	15,000
Events lost	**0**
Average latency	38.993 ms
Throughput	157.41 events/sec

All events were successfully processed even while one broker was unavailable.

```
PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practical\practical3> .\venv\
Scripts\Activate.ps1
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practical\practical3>
$env:CONSUMER_ID="fault-consumer"
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practical\practical3>
python streaming\spike_consumer.py

===== SPIKE CONSUMER =====
Consumer ID: fault-consumer
Consumer group: spike-consumer-group
Idle timeout: 5 seconds
Waiting for traffic...


===== SPIKE CONSUMER COMPLETE =====
Consumer ID: fault-consumer
Events processed: 15000
Average latency: 34.628 ms
Throughput: 137.87 events/sec
Metrics saved to: metrics/fault-consumer_spike_metrics.csv
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practical\practical3>
```

After the test, the failed broker was restarted and the topic was verified with **3 partitions and 3 replicas.**

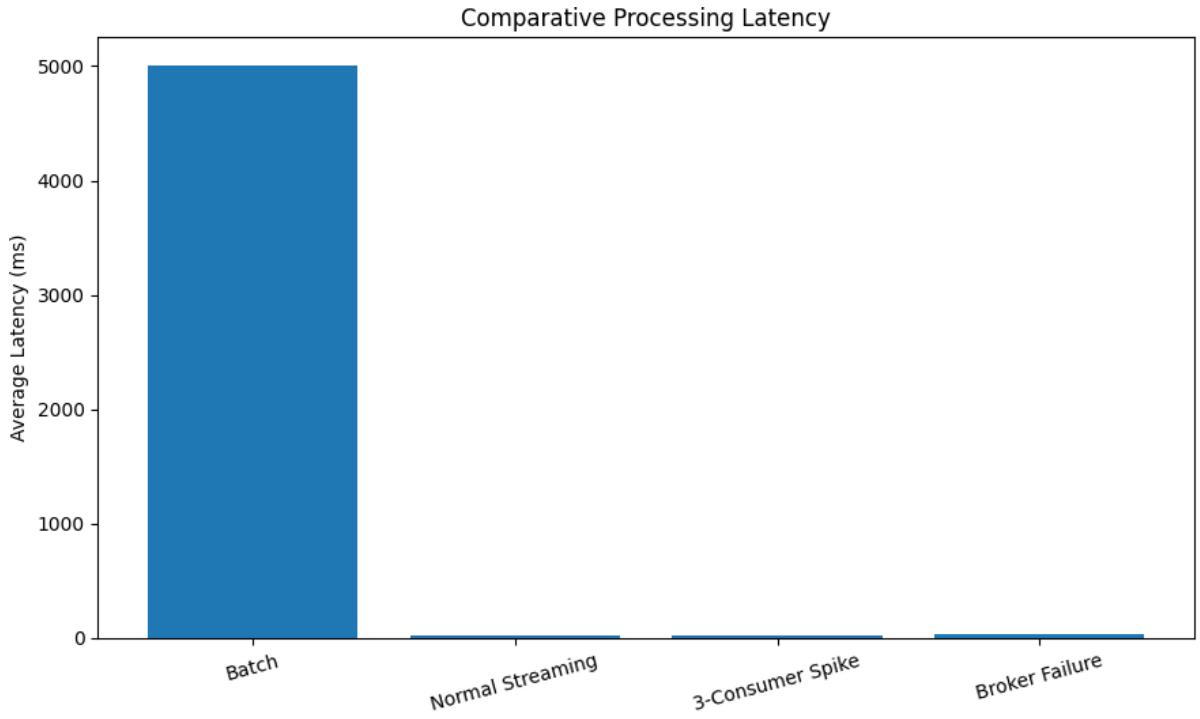
```
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practical\practical3> docker ps
CONTAINER ID   IMAGE                                          COMMAND                  CREATED        STATUS              PORTS
                                                NAMES
0955b8fa9286   docker.redpanda.com/redpandadata/redpanda:v25.2.1   "/entrypoint.sh redp…"   22 hours ago   Up About an hour   0.0.0.0:18083->18083/tcp, [::]:18083->18083/
tcp, 0.0.0.0:19093->19093/tcp, [::]:19093->19093/tcp   redpanda-1
8387e1fbfd2d   docker.redpanda.com/redpandadata/redpanda:v25.2.1   "/entrypoint.sh redp…"   22 hours ago   Up 22 hours        0.0.0.0:18082->18082/tcp, [::]:18082->18082/
tcp, 0.0.0.0:19092->19092/tcp, [::]:19092->19092/tcp   redpanda-0
c7aa62076728   docker.redpanda.com/redpandadata/redpanda:v25.2.1   "/entrypoint.sh redp…"   22 hours ago   Up 22 hours        0.0.0.0:18084->18084/tcp, [::]:18084->18084/
tcp, 0.0.0.0:19094->19094/tcp, [::]:19094->19094/tcp   redpanda-2
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practical\practical3> docker exec redpanda-0 rpk topic describe user-activity
SUMMARY
=======
NAME        user-activity
PARTITIONS  3
REPLICAS    3

CONFIGS
=======
KEY                                VALUE          SOURCE
cleanup.policy                     delete         DEFAULT_CONFIG
compression.type                   producer       DEFAULT_CONFIG
delete.retention.ms                -1             DEFAULT_CONFIG
flush.bytes                        262144         DEFAULT_CONFIG
flush.ms                           100            DEFAULT_CONFIG
initial.retention.local.target.bytes  -1          DEFAULT_CONFIG
initial.retention.local.target.ms  -1             DEFAULT_CONFIG
max.compaction.lag.ms              9223372036854  DEFAULT_CONFIG
max.message.bytes                  1048576        DEFAULT_CONFIG
message.timestamp.type             CreateTime     DEFAULT_CONFIG
min.cleanable.dirty.ratio          0.2            DEFAULT_CONFIG
min.compaction.lag.ms              0              DEFAULT_CONFIG
redpanda.iceberg.mode              disabled       DEFAULT_CONFIG
redpanda.leaders.preference        none           DEFAULT_CONFIG
redpanda.remote.allowgaps          false          DEFAULT_CONFIG
redpanda.remote.delete             true           DEFAULT_CONFIG
redpanda.remote.read               false          DEFAULT_CONFIG
redpanda.remote.write              false          DEFAULT_CONFIG
retention.bytes                    -1             DEFAULT_CONFIG
retention.local.target.bytes       -1             DEFAULT_CONFIG
retention.local.target.ms          86400000       DEFAULT_CONFIG
retention.ms                       604800000      DEFAULT_CONFIG
segment.bytes                      134217728      DEFAULT_CONFIG
segment.ms                         1209600000     DEFAULT_CONFIG
write.caching                      true           DEFAULT_CONFIG
(venv) PS C:\Users\Tarjani\OneDrive\Pictures\Documents\Notes_SEM5\DE\practical\practical3>
```

G. PERFORMANCE COMPARISON

Test	Events	Avg. Latency	Throughput
Batch	5,000	5,001.391 ms	99.973 events/s
Normal Streaming	5,000	17.719 ms	33.28 events/s
3-Consumer Spike	15,000	22.635 ms	—
Broker Failure	15,000	38.993 ms	157.41 events/s

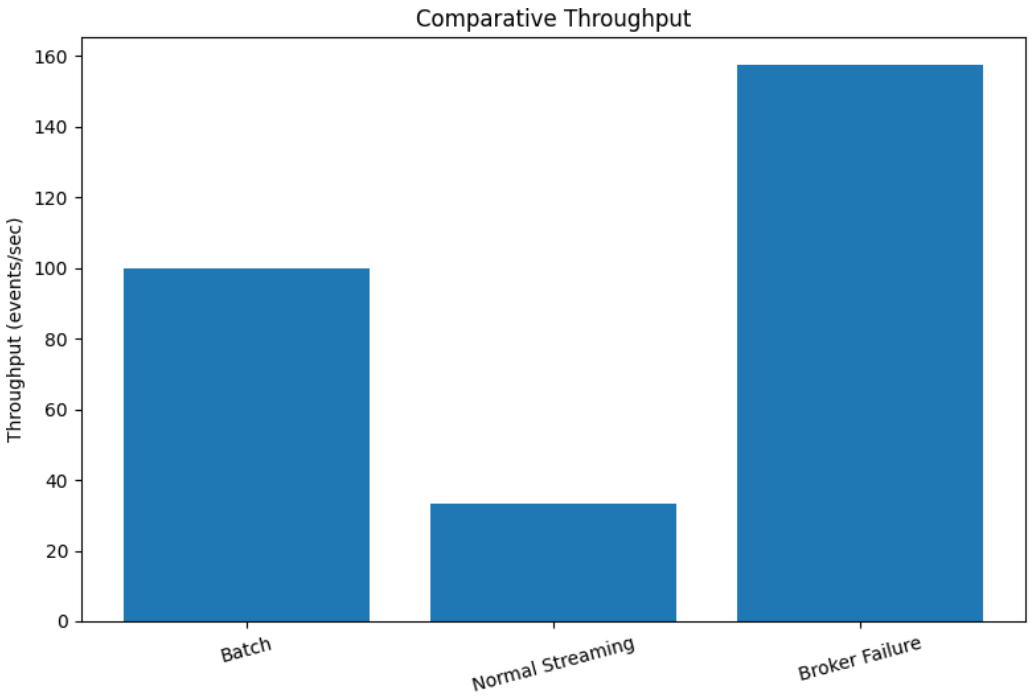
Comparative Latency



Observation

Batch processing showed significantly higher latency because events were processed periodically. Streaming maintained much lower latency even under the traffic spike and broker-failure test.

Comparative Throughput



Observation

The streaming system continued processing events during the broker failure, demonstrating that the replicated architecture could maintain operation despite one broker being unavailable.

10. KEY QUESTIONS

1. When is micro-batch better than true real-time streaming?

Micro-batch processing is useful when very low latency is not required. It is suitable for **historical transaction processing, scheduled reports, ETL and periodic analytics**, where simpler processing and bulk operations are preferred.

2. How does the streaming broker handle broker failure?

The topic uses **replication** across multiple brokers. In this experiment, the topic had **3 replicas**. When redpanda-1 was stopped, the remaining brokers continued serving the replicated data, and all **15,000 events were processed without loss**.

3. What are the network and compute trade-offs?

Batch processing reduces per-event overhead by processing records together, making it efficient for bulk operations. Streaming processes events continuously, giving lower latency but introducing per-event serialization, network and broker-processing overhead.

CONCLUSION

The practical successfully implemented both batch and real-time streaming ingestion.

The batch pipeline achieved approximately 99.97 events/sec but had a much higher latency of approximately 5 seconds. The streaming pipeline provided much lower latency, with 17.719 ms during normal operation.

The three-consumer test successfully distributed 15,000 events across the consumers. During the broker-failure test, all 15,000 events were processed with zero event loss, demonstrating the fault tolerance provided by replication.

Therefore, batch ingestion is suitable for bulk and historical processing, while real-time streaming is more suitable for applications requiring low latency, continuous processing and fault tolerance.